Circuits for Computing

Robert Y. Lewis

CS 0220 2021

February 7, 2022

Overview

Review of binary arithmetic

When we write numbers in base 10, each digit represents a power of 10:

$$405 = 5 \cdot 10^0 + 0 \cdot 10^1 + 4 \cdot 10^2$$

When we write numbers in base 2 (binary), each digit represents a power of 2:

$$10011 = 1 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4$$

$$10011_2 = 19_{10}$$

We refer to each digit as a “bit.” 10011 is a 5-bit number.

Why base 10? Why base 2?

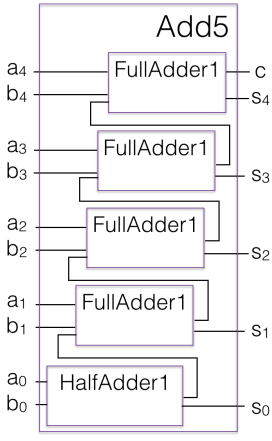
Cultures/languages that use Arabic numerals have generally settled on base 10 (decimal).

Computers really like base 2 (binary). (Also base 8, octal.)

Why?

5-bit-number adder

$$a = a_4 a_3 a_2 a_1 a_0$$
$$b = b_4 b_3 b_2 b_1 b_0$$



Negative numbers

We've defined addition in terms of logic gates. How about subtraction? We can compute subtraction via adding a negative number. What's a negative number in bits...?

$-x$ is a value such that $x + (-x) = 0$.

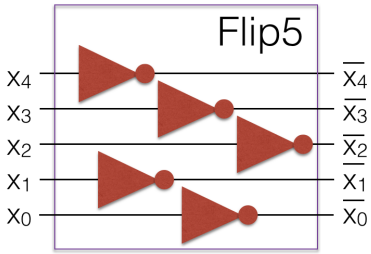
In computers, when we add two 5-bit numbers, we generally store a 5-bit answer, which means throwing away the carry. Thus, if we want the sum of two 5-bit numbers to result in a zero, we get that if they add to 32. So, to negate x, we can compute $32 - x$ (or just 0 if $x = 0$).

Example: $-5 \rightarrow 27$.

In binary: -00101 $\rightarrow$ 11011.
Progress? Well, we have a way to represent a negative number, but computing it seems to require subtraction, which is what we were trying to figure out in the first place...

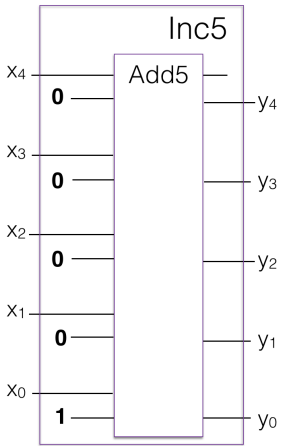
Bit flips

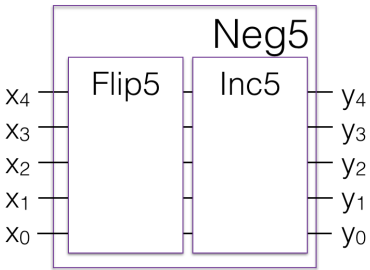
Define $\overline{x}$ to be the number x but with all its bits flipped.



Note: $x + \overline{x} = 2^5 - 1 = 31$ since the sum has every bit turned on. Thus, $\overline{x} = 31 - x$. If we add one to both sides, we have $\overline{x} + 1 = 32 - x$. And, $32 - x$ is the additive inverse, which is what we wanted to compute.

Negation by two's complement





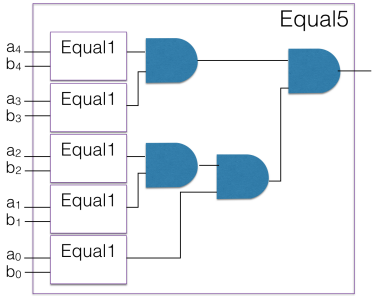
Positive and Negative Numbers

0	00000			
1	00001	11111	-1	(was 31)
2	00010	11110	-2	(was 30)
3	00011	11101	-3	(was 29)
4	00100	11100	-4	(was 28)
	...			
13	01101	10011	-13	(was 19)
14	01110	10010	-14	(was 18)
15	01111	10001	-15	(was 17)
		10000	-16	(was 16)

We interpret the numbers this way so we can use the first bit (high order bit) as the sign (0 for positive, 1 for negative).

Equal5

Can you design a circuit that takes in two 5-bit numbers $a = a_4a_3a_2a_1a_0$ and $b = b_4b_3b_2b_1b_0$ and outputs one bit: 1 if $a = b$, 0 otherwise?



More arithmetic circuits

The game we're playing: build up a "toolbox" of smaller circuits. Compose them together into larger ones. Once you know the behavior of a smaller circuit, you don't need to know its implementation.

Modular programming!

Pros/cons of special purpose circuits

Pros:

- Modular design
- Can do lots of things
- Can be *very* fast, depending on design

Cons:

- Cannot be repurposed
- Design can be difficult
- Need to plan for the “worst-case” application

Programmable circuits

Let's change our model. Design a very simple programming language. 8 commands, indexed by 3-bit numbers.

- Memory: RAM, an array of stored numbers
- Accumulator: one "special" stored number
- Commands: the program, stored as an array of numbers
- Arguments: numbers passed to commands, e.g. memory locations
- Program counter: indicates which command should be executed next

Each command can update the memory, accumulator, and program counter.

Commands: "add value in memory location *m* to accumulator," "set accumulator to value," "negate accumulator," "go to command if accumulator is positive," ...

Programmable circuits

Key idea: *all of these commands can be implemented as circuits!*

So we get a loop: a “main” circuit reads the next command, and directs its argument to the appropriate sub-circuit, which updates the memory, accumulator, and/or program counter. Rinse and repeat.

This is a computer!
Some slides at the end (optional—we won’t go through them in lecture) show off what these circuits might look like.

The Incredible Proof Machine

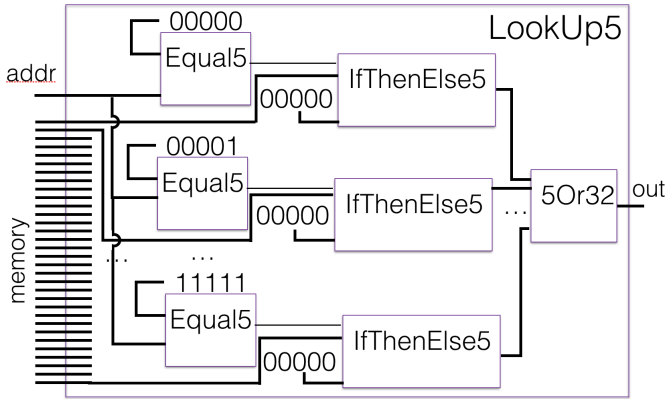
Here's a different angle on circuits in logic: The Incredible Proof Machine!

`https://incredible.pm/`

NOTE: Not the same kind of circuit as in the rest of the slides. Don't confuse the two.

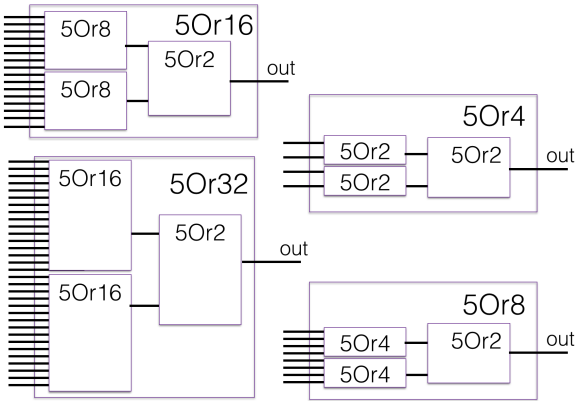
Lookup

LookUp5 takes a 5-bit address, $2^5 = 32$ 5-bit numbers and returns the 5-bit number at position “addr” of the list.



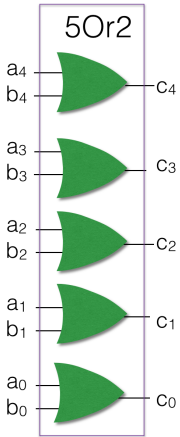
5Or*x*

LookUp5 combines 32 5-bit numbers into one. 5Or*x* takes *x* 5-bit numbers and ors them together bitwise.

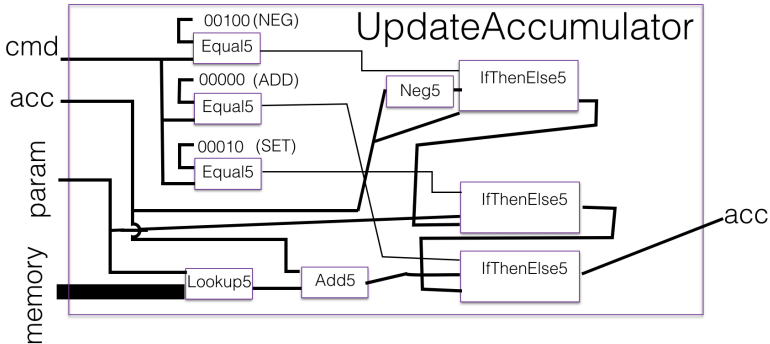


Bottom out

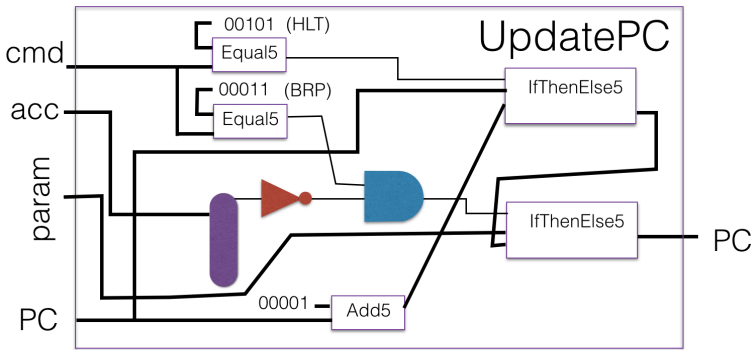
At bottom level, two 5-bit numbers are ored bitwise.



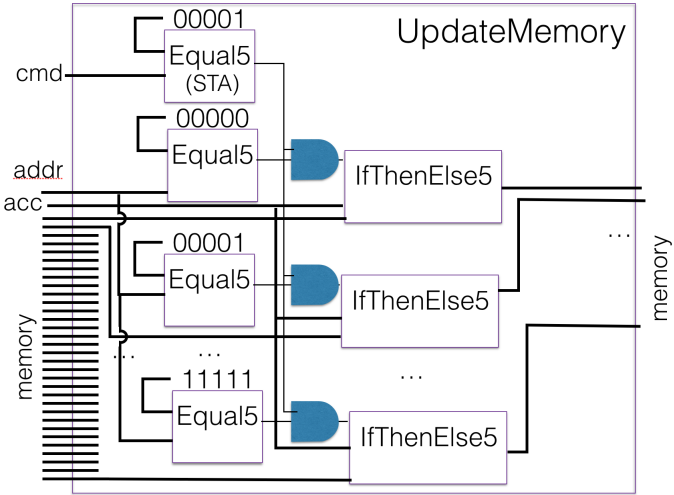
Accumulator update



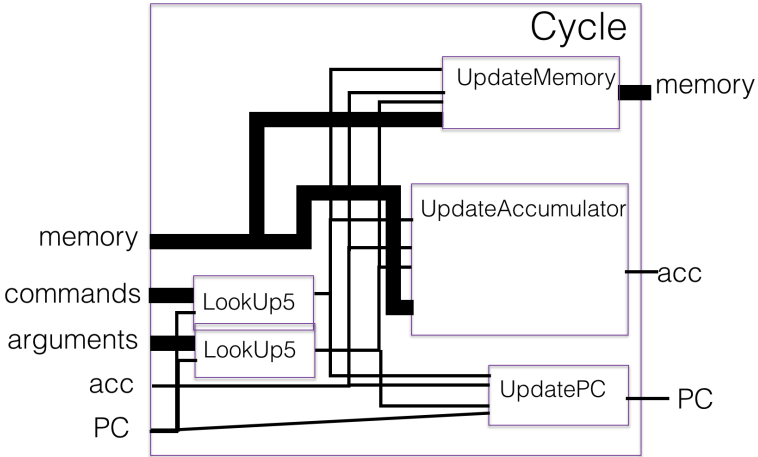
Program counter update



Memory update



Cycle


Cycle
UpdateMemory
memory
UpdateAccumulator
acc
memory
commands
LookUp5
arguments
LookUp5
acc
UpdatePC
PC
PC